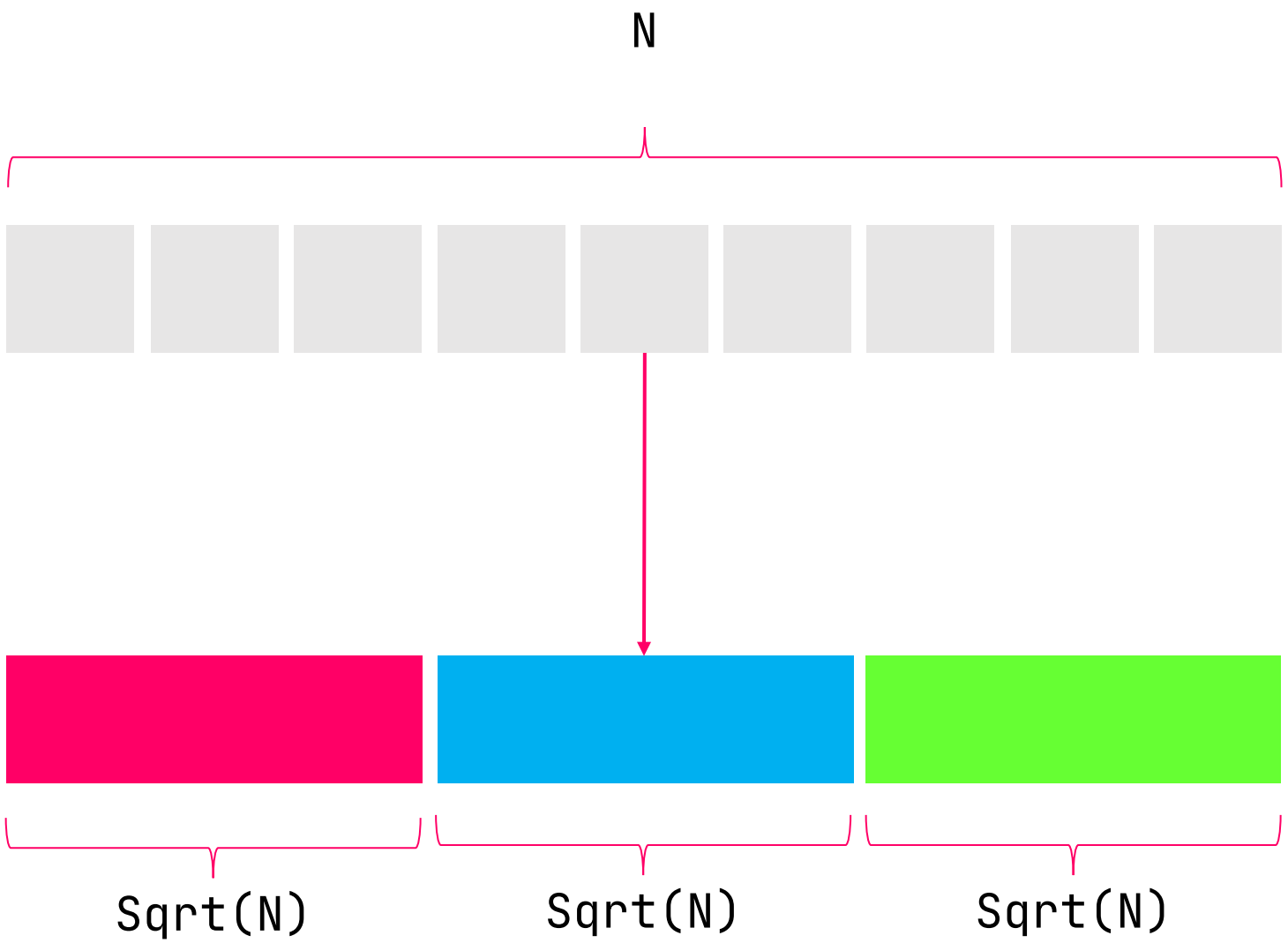
Range Query

Square Root Decompostion



Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Square Root Decompostion

```
const int MAX_N = 9 ;
int Size , clusterSize ;

int arr[MAX_N] = {5 , 6 , 3 , 9 , 8 , 2 , 6 , 4 , 8} ;
int cluster[ceil(sqrt(MAX_N))] ;
```

```
void build () {

    Size = ceil(sqrt(MAX_N)) ;
    clusterSize = ceil(MAX_N / Size) ;

    for(int i = 0 ; i < MAX_N ; i ++ ){
        cluster[i / Size] += arr[i] ;
    }
    return ;
}
```

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 1 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 1 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 1 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 1 - 7 = 6 + 3 + 19 + 6 + 4 = 38

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 4 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 4 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 4 - 7 ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 4 - 7 = 8 + 2 + 6 + 4 = 20

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

Sum 1 - 7 = 6 + 3 + 19 + 6 + 4 = 38

Square Root Decompostion

```
int query(int l , int r){

    int res = 0 ;

    while(l < r && l % Size ≠ 0){
        res += arr[l ++] ;
    }

    while(l + Size - 1 ≤ r && l % Size == 0){
        res += cluster[l / Size] ;
        l += Size ;
    }

    while(l ≤ r){
        res += arr[l ++] ;
    }
    return res ;
}
```

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	5	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	14	19	18

update(idx = 0 , val = 10) ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	10	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	0	19	18

update(idx = 0 , val = 10) ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	10	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	19	19	18

update(idx = 0 , val = 10) ?

Square Root Decompostion

idx	0	1	2	3	4	5	6	7	8
arr	10	6	3	9	8	2	6	4	8

idx	0	1	2
cluster	19	19	18

update(idx = 0 , val = 10) ?

Square Root Decompostion

```
void update(int idx , int val){

    arr[idx] = val ;

    int l = (idx / Size) * Size ;
    int r = min(MAX_N - 1 , l + Size - 1) ;
    int cluster_idx = l / Size ;

    cluster[cluster_idx] = 0 ;

    for(int i = l ; i ≤ r ; i ++ ){
        cluster[cluster_idx] += arr[i] ;
    }
    return ;
}
```

Square Root Decompostion

```
void build () {

    Size = ceil(sqrt(MAX_N)) ;
    clusterSize = ceil(MAX_N / Size) ;

    for(int i = 0 ; i < clusterSize ; i ++ ){
        cluster[i] INT_MAX ;
    }

    for(int i = 0 ; i < MAX_N ; i ++ ){
        cluster[i / Size] = min(cluster[i / Size] , arr[i]);
    }
    return ;
}
```

Square Root Decompostion

```
int query(int l , int r){

    int res = INT_MAX ;

    while(l < r && l % Size ≠ 0){
        res = min(res , arr[l ++ ]) ;
    }

    while(l + Size - 1 ≤ r && l % Size == 0){
        res = min(res , cluster[l / Size]) ;
        l += Size ;
    }

    while(l ≤ r){
        res = min(res , arr[l ++ ]) ;
    }

    return res ;
}
```

Square Root Decompostion

```
void update(int idx , int val){

    arr[idx] = val ;

    int l = (idx / Size) * Size ;
    int r = min(MAX_N - 1 , l + Size - 1) ;
    int cluster_idx = l / Size ;

    cluster[cluster_idx] = INT_MAX ;

    for(int i = l ; i ≤ r ; i ++ ){
        cluster[cluster_idx] = min(cluster[cluster_idx] , arr[i]) ;
    }
    return ;
}
```

Square Root Decompostion

Datastructure	Size	Build()	Query()	Update()	RangeUpdate()
Quicksum	N	O(N)	O(1)	O(N)	O(N)
Segment Tree	2N	O(N)	O(log(N))	O(log(N))	O(N log(N))
Fenwick Tree	N	O(N)	O(log(N))	O(log(N))	O(N log(N))
Sqrt Decomposition	Sqrt(N)	O(N)	O(Sqrt(N))	O(Sqrt(N))	O(N Sqrt(N))

CSES Dynamic Range Sum Queries

[30 min]

Start

CSES Dynamic Range Minimum Queries

[30 min]

Start

Square Root
Decompostion

o61_oct_c1_charprint

[30 min]

Start

TOI17 : Sequel (with Sqrt-De)

[60 min]

Start